ARC: Accurate, Real-Time, and Scalable Multi-Vehicle Cooperative Perception

Kaleem Nawaz Khan
Rochester Institute of Technology
Rochester, NY, USA
kk5271@rit.edu

Fawad Ahmad
Rochester Institute of Technology
Rochester, NY, USA
fawad@cs.rit.edu

Abstract

To overcome line-of-sight limitations in 3D sensors, cooperative perception shares sensor information between vehicles in real time. Core to cooperative perception is the alignment of sensor data in multiple coordinate systems. Existing techniques use 3D maps and GPS for alignment, but these can lead to inaccurate alignments. However, improving alignment accuracy incurs additional compute latency, which is not desirable. In this paper, we present ARC[1], a system that carefully navigates the trade-off between latency and accuracy for point cloud alignment. ARC uses an anchor-based alignment technique to align vehicles to a common coordinate system. It minimizes latency by using grid-based spatial reasoning to perform alignment only in overlapping regions of the point clouds. ARC reuses the same spatial reasoning to selectively share only the most relevant data among vehicles. ARC, on traces collected from the real world and simulations, can fuse point clouds from up to 40 vehicles with an accuracy of under 7 cm and achieve a mean compute latency of 20 ms.

CCS Concepts

• **Computing methodologies** → **Cooperation and coordination**; • **Networks** → **Sensor networks**.

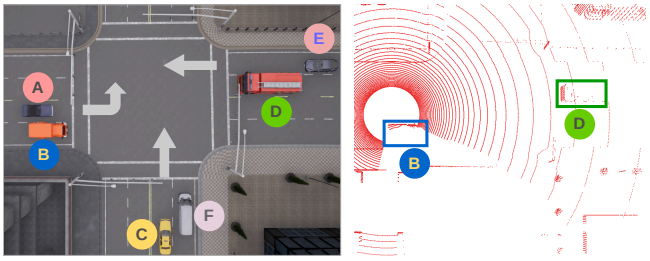
Keywords

Autonomous Vehicles, Cooperative Perception, Sensor Fusion, Point Cloud Alignment, Collaborative Sensing

ACM Reference Format:
Kaleem Nawaz Khan and Fawad Ahmad. 2026. ARC: Accurate, Real-Time, and Scalable Multi-Vehicle Cooperative Perception. In *ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys '26), May 11–14, 2026, Saint Malo, France.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3774906.3800484

1 Introduction

Critical to the operation of autonomous vehicles is their ability to understand the environment around them. For this, they use sensors such as cameras [9], radars [44], and LiDARs [31]. LiDARs send out millions of light rays multiple times per second and, from their returns, construct 3D point clouds. These are data structures that consist of a large number of points, each defined by its 3D position along with other attributes such as intensity and reflectivity. However, LiDARs are prone to occlusions and line-of-sight limitations. This is especially prevalent at traffic intersections where occlusions are common due to the dense traffic flow. Intersections account for nearly half of all fatal accidents in the U.S. [30].



(a) A turns left; others go straight. (b) LiDAR point cloud of vehicle A.

Figure 1: Bird's-eye view of an intersection: (a) vehicle A turns left while the other vehicles go straight. (b) In vehicle A's LiDAR point cloud, vehicles B and D are clearly visible, whereas vehicles C and F are occluded by vehicle B, and vehicle E is occluded by vehicle D.

Cooperative perception solves this challenge by having vehicles (and road sensors [21]) share 3D point clouds with each other [55]. For instance, in Fig. 1, vehicle A cannot perceive vehicles C, F, and E because they are occluded by vehicles B and D. If vehicles B and D share their point clouds with vehicle A, it can overcome this occlusion. After receiving the point clouds from vehicle B or vehicle D, vehicle A aligns the point cloud to its own coordinate system. Then, it appends the aligned point clouds to its own to build a fused point cloud (Fig. 2). This is called cooperative perception.

While cooperative perception is conceptually straightforward, applying it to autonomous (or human) driving imposes strict latency and accuracy requirements. Autonomous vehicles must sense their surroundings, plan a path, and actuate at least 10 times per second with a tail latency below 100 ms [7, 26]. Moreover, they must localize themselves and nearby objects with centimeter-level accuracy [5]. Naturally, these constraints apply to cooperative perception as well.

Accurate and fast cooperative perception poses several system-level challenges. Vehicles must share and align point clouds at low latency, leaving ample time for downstream perception modules (object detection, motion forecasting, etc.) to consume the fused data. The fused data must also be correct, *i.e.,* the 3D point clouds must be aligned accurately. Lastly, this capability should scale to a large number of vehicles. However, low latency and high accuracy are inherently conflicting goals due to network and compute constraints. Point cloud alignment is compute-intensive and

[1]GitHub Repository: https://github.com/nsslofficial/ARC

SenSys '26, Saint Malo, France

ACM ISBN 979-8-4007-2309-4/26/05
https://doi.org/10.1145/3774906.3800484

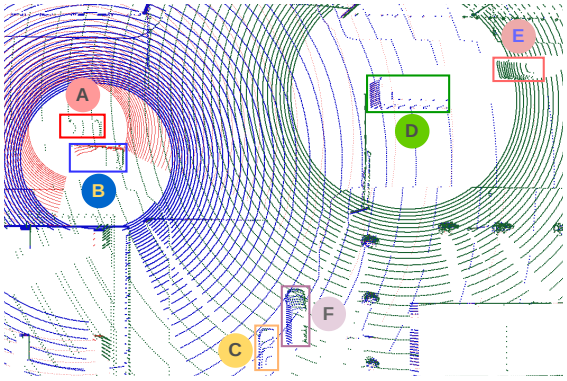


Figure 2: After fusing the point clouds from vehicle B (blue) and vehicle D (green) with its own (red), vehicle A gains visibility of all.

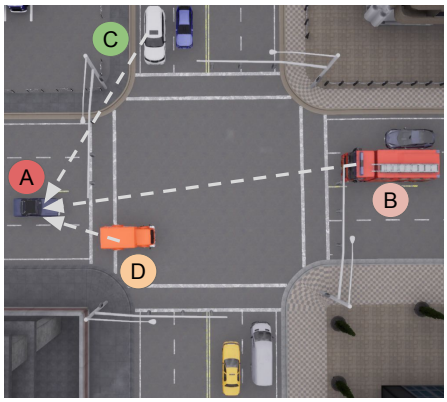


Figure 3: All vehicles align with the anchor (vehicle A).

can be erroneous. Moreover, because 3D point clouds are voluminous [35, 46], sharing multiple point clouds over the wireless network can easily overwhelm it. Although prior works [34, 55] address network bottlenecks by exchanging only the most relevant information among vehicles, *to our knowledge, achieving accurate alignment across multiple vehicles remains relatively unexplored.*

To better understand, we describe how point cloud alignment is performed. The first set of techniques, called direct alignment, matches overlapping regions between point clouds (and the features in them) to compute a transformation matrix. This transformation matrix, when applied to one point cloud, aligns it to the other's coordinate system. If a vehicle has to align with multiple vehicles, it must directly align with every vehicle. For instance, if a vehicle needs data from four vehicles, it might directly align with all four vehicles. Consequently, this approach does not scale. To this end, prior works [33, 34] use indirect alignment, *i.e.,* they align all vehicles to a shared 3D map[2]. Because every vehicle aligns itself to the same 3D map, they indirectly align with one another. However, 3D maps can contain noise because they are built by aligning thousands of point clouds [25]. *Noise in the 3D map can significantly degrade point cloud alignment accuracy.*

Both approaches exhibit trade-offs. Direct alignment is accurate but incurs significant latency and does not scale well. Indirect alignment, on the other hand, is fast and scalable but suffers from lower accuracy. In this paper, we ask the question: *Can we design a cooperative perception system that enables high-accuracy alignment at low latency and scales to a large number of vehicles?*

Our Approach. To simultaneously achieve high accuracy and low latency, we propose an anchor-based alignment technique. Instead of directly aligning with each other, vehicles align themselves with a common anchor. This anchor can be a LiDAR mounted on the roadside or a moving vehicle. Once all the vehicles align themselves with the anchor, they are indirectly aligned with one another. For example, in Fig. 3, vehicle A is the anchor with which the other vehicles (B, C, and D) align themselves.

Anchor-based alignment has two benefits. Although it is an indirect alignment, it achieves high accuracy. This is because vehicles align themselves to a single LiDAR frame, as opposed to an accumulation of LiDAR frames (a 3D map), which can be prone to noise. Second, it ensures low latency because vehicles only need a single alignment operation, *i.e.,* with the anchor point cloud.

Challenges. While anchor-based alignment addresses the limitations of prior techniques, it introduces challenges of its own.

- Every vehicle must align its point cloud with the anchor's point cloud at every frame. This point cloud alignment must be fast to allow the downstream perception modules to consume the fused point cloud. At the same time, the alignment must be accurate.
- Second, once aligned, if all vehicles at the intersection share their raw point clouds, the wireless network could easily be overwhelmed. So, vehicles should only exchange relevant 3D data. Determining which data is relevant can be computationally intensive.

Contributions. We tackle these challenges by building an end-to-end system, ARC. It makes the following contributions.

- For fast alignment, instead of aligning the entire point clouds, we compute the overlapping regions between pairs of point clouds and align only those regions. To do this, we propose a fast spatial reasoning module that uses an underlying shared 3D grid to determine the overlapping regions between the point clouds.
- Instead of using a separate module to determine what data to send to each vehicle, we reuse the 3D grid. By doing so, we enable accurate alignment without trading off compute latency. Moreover, we design the 3D grid so that operations on it can be easily offloaded to the GPU. This enables us to identify overlapping regions and blind spots almost instantly.

An end-to-end implementation of ARC aligns vehicles with up to cm-level accuracy at an end-to-end compute latency of 20 ms.

2 Background and Motivation

Limitations of 3D Sensors. Autonomous vehicles are equipped with depth perception sensors like LiDARs. A LiDAR consists of several radially positioned laser beams. Together, these laser beams emit millions of light rays per second and, from their reflections, build a 3D point cloud of the environment. This point cloud consists of 3D points, defined by their positions and other attributes like intensity and reflectivity. However, because LiDARs build point clouds from reflected light rays, like human vision, they are prone to occlusions and line-of-sight limitations. For instance, vehicle A's

[2]This is a large point cloud that has a 1:1 correspondence with the physical world and vehicles use this to localize themselves in the world.

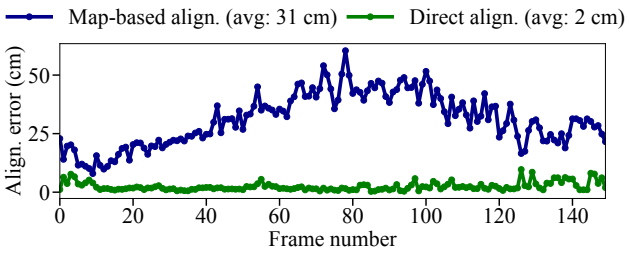


Figure 4: Indirect alignment (align.) through a 3D map incurs an order-of-magnitude higher alignment error than direct alignment.

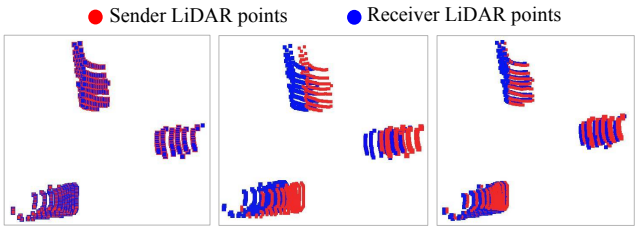
point cloud does not contain reflections from vehicle C because it is occluded by vehicle B (Fig. 1). Additionally, the radial arrangement of a LiDAR's beams causes point density to decrease with distance; the density of points (or reflections) drops significantly with distance. In other words, the number of reflected points from an object of the same size placed further away from the LiDAR is smaller than that of objects nearby. Consequently, object detectors [22, 23] may not accurately recognize objects further away from the LiDAR.

Cooperative Perception. To extend a vehicle's perception range, cooperative perception shares 3D point clouds between vehicles (and roadside sensors) over the wireless network. To do this, at every frame, one vehicle (the sender) transmits its point cloud over the wireless network to the other vehicle (the receiver). Upon receiving the point cloud, the receiver aligns it in its own coordinate system. After alignment, the receiver appends the aligned point cloud to its own to build a fused point cloud. Then, downstream perception modules consume the fused point cloud.

Requirements for Cooperative Perception. An autonomous vehicle must sense, plan, and react to its surroundings at least 10 times per second, with a tail latency below 100 ms [26]. Moreover, it must localize itself and surrounding objects with up to cm-level accuracy. Consequently, cooperative perception must also be both fast and precise. If cooperative perception cannot build the fused point cloud with low latency, by the time downstream perception modules consume the data, it will already be stale. Similarly, if point clouds are not aligned to cm-level accuracy, it can be catastrophic for downstream modules (such as scene understanding and planning), which will operate on incorrect data.

Broadly, cooperative perception approaches can employ early or late fusion. Late fusion reduces network bandwidth by transmitting compact, task-specific features (e.g., bounding boxes). However, aligning such compact features is often inaccurate and produces a fused representation that is unsuitable for all downstream applications [39]. Early fusion may consume more bandwidth because it transmits raw point clouds, but the alignment is more accurate [21]. Furthermore, the resultant fused point cloud can be readily processed by downstream perception and planning modules.

Point Cloud Fusion. Point cloud fusion is a two-step process: alignment and stitching. Given point clouds P_a and P_b from vehicles A and B, the goal of point cloud alignment is to find a rigid transformation T_{a-b} that aligns P_a with P_b. The transformation T_{a-b} is a 4x4 matrix, consisting of a rotation matrix R and a translation vector t. This transformation T_{a-b}, when applied to P_a, yields



(a) Ground-truth alignment (b) Indirect alignment (error: 30 cm) (c) Direct alignment (error: 5 cm)

Figure 5: Qualitative impact of alignment error on the fused point cloud from sender and receiver: direct alignment error (5 cm) is barely visible, while indirect alignment error (30 cm) is noticeable.

a transformed point cloud $P_a^{'}$ in the same coordinate system as P_b. The stitching simply appends $P_a^{'}$ to P_b to obtain a fused point cloud P_f that contains all the points from $P_a^{'}$ and P_b.

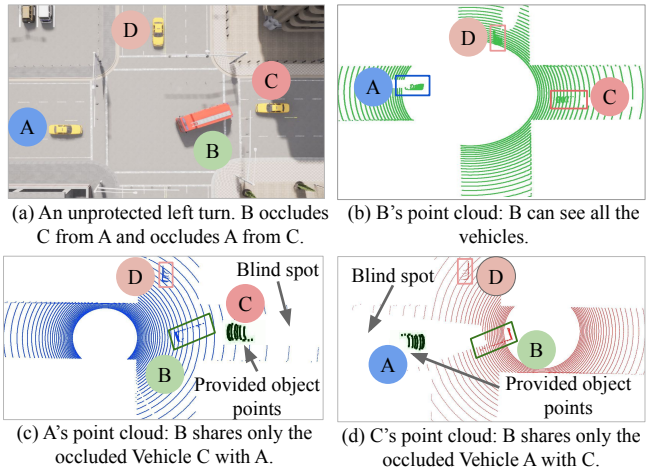
Iterative Closest Point (ICP [36]). Point cloud alignment algorithms, such as ICP, iteratively compute a transformation matrix T_{a-b} that minimizes the 3D distance from every point in P_a to its nearest neighbor in P_b. To do this, ICP requires both a large overlap between the two point clouds and a coarse-grained alignment between them. This coarse-grained alignment is an accurate initial guess for the transformation matrix T_{a-b}. If the two vehicles are far away and/or do not have an accurate initial alignment, ICP will not be able to accurately align the two point clouds.

Direct Vs Indirect Alignment. Direct alignment feeds the two point clouds (P_a and P_b) to an alignment algorithm, such as ICP, to compute a transformation matrix (T_{a-b}). Although direct alignment is accurate, it can incur significant latency if a vehicle has to align with 3D point clouds from multiple vehicles. For instance, in Fig. 2, to get a complete scene understanding, vehicle A needs to align with two different vehicles. This entails two direct alignments. To tackle this, prior works use indirect alignment.

Indirect alignment aligns both point clouds to a third point cloud (*e.g.,* a 3D map). This produces two transformation matrices (T_{a-m} and T_{b-m}). Using these matrices, we can compute the alignment between P_a and P_b as $T_{a-b} = T_{a-m} * T_{b-m}^{-1}$. However, because a 3D map can contain noise, this can lead to inaccurate alignments.

To demonstrate this, we collected LiDAR traces from two vehicles and then aligned them using direct and indirect alignment. Fig. 4 shows that the error for indirect alignment (Map-based align.) can be an order of magnitude greater than direct alignment (Direct align.). Qualitatively, Fig. 5 illustrates the effect of alignment error on the fused point cloud. A 5 cm error (for direct alignment) is hardly noticeable, whereas a 30 cm error (indirect alignment) can cause misdetections and errors in downstream perception modules. In this paper, we ask the question: *How can we achieve accurate alignment at low latency for a large number of vehicles?*

Network Bottleneck. A LiDAR sensor such as Hesai OT128 [42], a 128-beam LiDAR with a 200 m range and 360° field of view, can produce up to 6.91 million points per second, corresponding to data rates of around 1 Gbps. Sending such high-bandwidth data over a wireless link can easily exhaust the available network capacity. To tackle this, prior works [34, 55] share only relevant information



(a) An unprotected left turn. B occludes C from A and occludes A from C.

(b) B's point cloud: B can see all the vehicles.

(c) A's point cloud: B shares only the occluded Vehicle C with A.

(d) C's point cloud: B shares only the occluded Vehicle A with C.

Figure 6: Vehicle B shares only relevant dynamic object points: it shares points of vehicle A to C, and points of vehicle C to A.

with other vehicles. They extract dynamic objects and then determine the subsets of those that are relevant to every vehicle. For instance, in Fig. 6a, B occludes A and C from each other. Although B can see all vehicles at the intersection (Fig. 6b), it will only share occluded vehicle C with A (Fig. 6c), and A with C (Fig. 6d).

However, determining what information is relevant can be inaccurate and inefficient. Decentralized approaches (AutoCast [34]) rely on the sender to estimate what the receiver cannot see. Consequently, they can make inaccurate determinations [55], and since there is no coordination among vehicles on what to share, multiple sender vehicles might end up sharing duplicate data with the receiver vehicle (resulting in higher bandwidth consumption). Centralized approaches (RAO [55]), on the other hand, require each vehicle to compute and share visibility maps or other large data structures to inform others of their blind spots, incurring additional bandwidth and computational overhead.

3 ARC Design

Overview. Consider an ARC deployment at an intersection. There can be two types of dynamic objects: compliant and non-compliant. Non-compliant objects include vehicles, pedestrians, and cyclists that do not participate in cooperative perception. Compliant objects participate in cooperative perception and are equipped with a LiDAR, computing resources, and a wireless radio. These objects can assume one or more of the following roles: anchor, producer, and consumer. All compliant objects spatially align with the anchor, which can be either a roadside-mounted LiDAR or a vehicle. Producers share their 3D point clouds with consumers to extend their perception. Moreover, we assume that all vehicles (including the anchor) share a semantic 3D map[3], where each 3D point, besides its position, has a semantic label associated with it (a common practice in the autonomous driving industry [2, 3, 15, 59]). This label defines the object class of the point (*e.g.*, vehicle, road, building, *etc.*).

[3]Existing approaches such as SuMa++ [11] build semantic 3D maps by projecting 2D image segmentation results onto LiDAR points or by directly applying 3D segmentation models such as RangeNet++ [28].

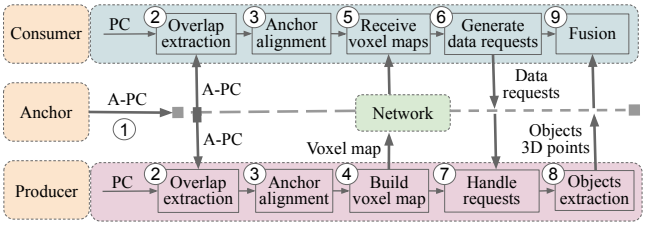


Figure 7: Overview of ARC: the anchor broadcasts its point cloud to establish a reference frame. Other vehicles (producer or consumer) align overlapping regions with it. They construct and exchange voxel maps. Consumers select producers and request relevant data. The received data is fused with local point clouds.

All vehicles (including the anchor) localize themselves in the 3D map. After localizing itself, as illustrated in Fig. 7, the anchor broadcasts its point cloud (A-PC) to other vehicles (step 1 in Fig. 7). Upon receiving the anchor point cloud, other vehicles align their point clouds (PC) to it. Because point cloud alignment can be compute-intensive, ARC finds and aligns only the overlapping regions of the point clouds (steps 2 and 3 in Fig. 7). ARC uses a fast overlap estimation algorithm that operates on a 3D grid data structure that vehicles share at the intersection. Once aligned to the anchor, vehicles can share 3D data with one another.

When sharing 3D data, ARC shares only relevant data to save network bandwidth. For this, every vehicle builds a compact scene representation on the shared 3D grid. We refer to this scene representation as the voxel map. In this representation, each vehicle declares the grid elements that are visible and occluded to it. Every vehicle then broadcasts this compact representation (step 4 in Fig. 7). Upon receiving these representations from other vehicles, each vehicle identifies other vehicles that have visibility into its occluded regions (step 5 in Fig. 7). The vehicle requesting the data is called a consumer, and the one providing the data is called a producer. The consumer requests the producer to send 3D data about those occluded regions (step 6 in Fig. 7). The producer handles requests from a single or multiple consumers (step 7 in Fig. 7). It extracts the object points in the requested grid elements and then sends them to the consumers (step 8 in Fig. 7). Once the consumer receives the data, it positions and fuses it into its own point cloud, and then downstream perception modules consume the data (step 9 in Fig. 7).

If the intersection has a roadside unit with a LiDAR, ARC sets it as the anchor. Otherwise, ARC uses a moving vehicle as the anchor. ARC determines the anchor vehicle based on proximity to the center of the intersection. To do this, after localizing themselves in the map, all vehicles broadcast their 3D positions. Then, at fixed time intervals[4], each vehicle computes the 3D distance of itself and all other vehicles from the center. The vehicle closest to the center is selected as the anchor. In rare cases where multiple vehicles are equally close, ARC breaks the tie by comparing the x-coordinate and, if needed, the y-coordinate of their positions. Once selected, the anchor vehicle begins broadcasting point clouds to other vehicles. When the anchor leaves the intersection, ARC selects a new anchor.

[4]We set this interval to half the average time vehicles take to cross the intersection.

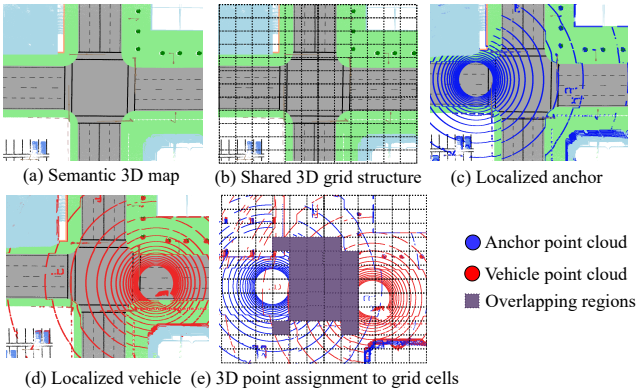


Figure 8: Utilizing the semantic 3D map (used by vehicles for localization) and a shared 3D intersection grid, ARC computes overlapping regions between the anchor and vehicle point clouds.

3.1 Fast Overlap Extraction

The Problem. To reduce latency, ARC uses anchor-based alignment. In this, all vehicles at the intersection align with an anchor (a vehicle or a roadside-mounted sensor). Point cloud alignment is computationally intensive and incurs high latency. A single alignment operation can take up to 30 ms for point clouds generated from a 128-beam LiDAR. This is not desirable as it does not leave ample time for downstream modules to process fused point clouds.

Key Insight. Alignment algorithms such as ICP compute a transformation matrix that aligns two point clouds as follows. First, for every point in one point cloud, ICP finds the closest point in the second point cloud. Second, it computes the 3D distances between corresponding points. Third, it computes a transformation matrix that minimizes these distances and iterates again. The accuracy and latency of ICP depend on two factors: (a) a coarse-grained alignment and (b) the overlapping region between the point clouds. A coarse-grained alignment is advantageous because (a) it reduces the chances of ICP converging to local minima, and (b) it reduces the number of iterations required for convergence. Building on prior work [21], we use vehicles' positions in the map as an initial guess for alignment. These need not be accurate, as their purpose is to reduce the search space for ICP. Lastly, finding nearest neighbors for all points can be computationally intensive. However, overlapping regions (or common regions) between two point clouds are often relatively small. Therefore, computing the transformation matrix requires only the overlapping points, not the entire point cloud.

Our Approach. ARC uses only the overlapping regions of two point clouds for alignment. Before aligning the point clouds, ARC estimates the overlapping regions between them. Then, it computes the transformation matrix for the overlapping regions and applies it to the entire point cloud. It is important to note that using overlapping regions for point cloud alignment is not novel to ARC. Prior works have computed and aligned overlapping regions [21, 40]. However, computing the overlap has traditionally been compute-intensive. To this end, ARC proposes a novel and efficient algorithm (Algorithm 1) to compute the overlap between two point clouds.

Algorithm 1 Fast Overlap Extraction

Input: $P_{\text{vehicle}}, P_{\text{anchor}}$, 3D grid G
Output: Cropped overlapping regions

1: initialize cell assignment map M
2: **for** $p \in P_{\text{vehicle}}, P_{\text{anchor}}$ **do**
3: $c \leftarrow$ nearest cell in G {assign point to cell}
4: append p_{vehicle} to $M[c]_{\text{vehicle}}$ and p_{anchor} to $M[c]_{\text{anchor}}$
5: **end for**
6: **for** each cell $c \in G$ **do**
7: $\rho_{\text{vehicle}} \leftarrow |M[c]_{\text{vehicle}}|/Size_{\text{cell}}$
8: $\rho_{\text{anchor}} \leftarrow |M[c]_{\text{anchor}}|/Size_{\text{cell}}$
9: $\rho_{\text{int}} \leftarrow \min(\rho_{\text{vehicle}}, \rho_{\text{anchor}})$ {interpoint density}
10: **if** $\rho_{\text{int}} >$ threshold **then**
11: mark c as overlapping {select cells for ICP}
12: **end if**
13: **end for**
14: crop points in selected cells from P_{vehicle} and P_{anchor}

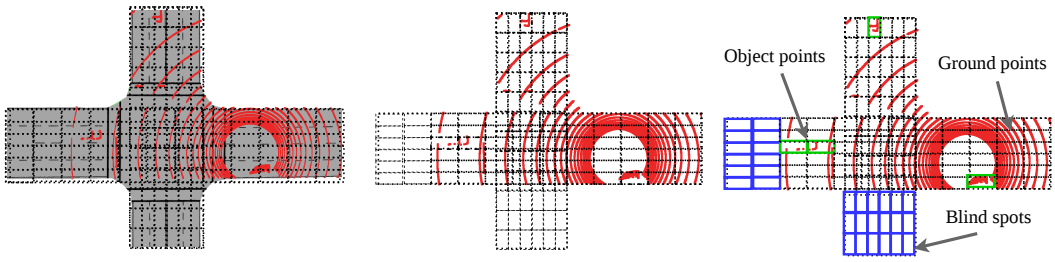
As illustrated in Fig. 8, all vehicles (including the anchor) share the map defined in §3, where each point is annotated with its object-class label (Fig. 8a). Offline, ARC overlays a 3D grid on the map (Fig. 8b). Each vehicle (including the anchor) matches its point cloud with the map to determine its position (Fig. 8c and Fig. 8d). At each frame, the anchor broadcasts its point cloud. Upon receiving it, vehicles project both their own and the anchor's point clouds onto the grid using their positions and the anchor's position (Fig. 8e).

After projecting both point clouds onto the grid, ARC employs Algorithm 1 to find the overlapping regions. ARC maintains a dictionary that records, for each point, the index of the grid cell to which it has been assigned (Algo. 1, lines 1-5). After this assignment, for both point clouds, ARC computes the point density in each grid cell (Algo. 1, lines 6-8). The point density is the number of points within the cell divided by the cell's size [5]. From the point densities, ARC computes interpoint density for each cell (Algo. 1, line 9). Interpoint density is the MIN of the point densities between the anchor and vehicle points. Prior work [21] shows that a large number of cells with high interpoint density indicates a large overlap, which in turn implies a higher chance of accurate alignment.

Next, ARC filters grid cells with high interpoint density (Algo. 1, lines 10-13). From each point cloud, ARC crops the points belonging to these high-density cells (Algo. 1, line 14), which form the overlapping region. ARC inputs these points to ICP and then applies the computed transformation matrix to the entire point cloud.

Since finding the overlapping regions is highly parallelizable, we offload this operation to the GPU. In our implementation, we first create a list of 3D points corresponding to grid cell centers. Using this list, each GPU thread independently performs a nearest-neighbor search. In this search, each point in the point cloud is assigned to its nearest cell center. Using the same grid, the GPU processes both anchor and vehicle point clouds independently. Then, for each grid cell, we compute the point density for both anchor and vehicle points separately. Using these densities, we compute interpoint densities for each cell. Finally, we extract two sets of points (one from the anchor and one from the vehicle) belonging

[5] We used a grid cell size of 2 × 3 × 4 meters. A larger cell size reduces latency but also lowers accuracy.



(a) Drivable space in the localized vehicle point cloud (b) Assignment of 3D points to grid cells (c) Generation of the voxel map

Figure 9: The process of generating a voxel map representation from a vehicle's point cloud using the shared 3D grid structure.

to cells with high interpoint densities. These sets constitute the dense overlapping regions of the two point clouds. Now, instead of using the entire point clouds for alignment, we use only these sets of points with GPU-accelerated ICP. This highly parallelized implementation allows our system to compute overlapping regions, crop them, and align point clouds within a few milliseconds.

3.2 Relevant Data Extraction

The Problem. Wireless network bandwidth for vehicular communication is limited. To this end, prior works [34, 55] extract and share only the 3D points of relevant dynamic objects in the scene. Determining which objects are relevant can be error-prone and latency-intensive. AutoCast [34], a decentralized approach, requires the sender vehicle to *estimate* what the receiver needs. This has two consequences: (a) the sender may inaccurately estimate the receiver's needs [55], and (b) multiple senders might transmit duplicate data to the receiver. We demonstrate these limitations in §4.4. Centralized approaches, such as RAO [55], explicitly request the relevant 3D points from sender vehicles. To do this, they construct and transmit data structures (occupancy maps) that describe what each sender sees. These data structures are expensive to compute and transmit over the wireless network. Similarly, to improve alignment accuracy, ARC introduces additional computations to extract and align overlapping regions. This adds to the system's end-to-end latency. ARC must address the high latency from prior work as well as the additional latency it introduces, so that downstream modules have ample time to process the fused point clouds.

Our Approach. To avoid sharing redundant data, in ARC, the receiver (consumer) vehicle explicitly requests data from one or more sender (producer) vehicles. For this, each vehicle broadcasts what it sees. In doing so, it introduces computational and network overhead. To reduce this, ARC reuses the shared 3D grid that vehicles use to align with the anchor. Each vehicle identifies a set of indices, indicating the grid cells in which it has visibility (*i.e.,* cells where the point density exceeds a predefined threshold).

At every frame, all vehicles (including producers, consumers, and the anchor) extract drivable space from their point cloud (Fig. 9a). Drivable space is the part of the environment on which a vehicle can legally drive, *e.g.,* the road surface. As described in §3.1, the map has semantic labels for every 3D point to describe the type of object it belongs to. Because each vehicle is aligned to the map, ARC can project semantic labels from the map onto the vehicle's point cloud and extract its drivable space.

Next, we compute the blind spots for every vehicle. This requires spatial reasoning over the 3D point clouds, which can be expensive. Instead, we reuse computations from ARC's overlap extraction module described in §3.1. Recall that every vehicle assigns 3D points from its point cloud to the shared grid (Fig. 9b). We mark grid cells where the point density is below a threshold, within the vehicle's LiDAR depth range, as blind spots. Blind spots are regions of the point cloud where the vehicle has little or no visibility.

After filtering blind spots, two types of grid cells remain: cells with objects and cells without objects. Since ARC focuses on dynamic objects, it identifies the cells with objects. We distinguish between the two types of cells. For the second class (*i.e.,* cells without objects), the LiDAR point cloud contains reflections from the ground plane. While ARC could use plane-fitting algorithms (e.g., RANSAC [17]) and clustering to identify these cells, this approach is computationally expensive. Instead, ARC leverages the map.

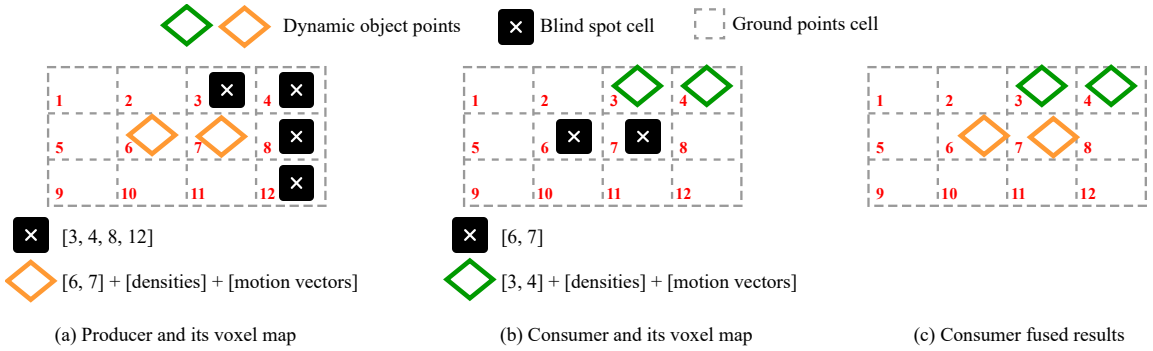
ARC subtracts the drivable space in the map at the intersection from the vehicle's drivable space. After this subtraction, the vehicle's point cloud contains only 3D points above the road surface. These points belong to dynamic objects. Because the vehicle's point cloud has already been projected onto the shared 3D grid, this grid now has two types of cells: a) blind spot cells (Fig. 9c: blue boxes) and b) cells with dynamic objects (Fig. 9c: green boxes).

Next, we compute motion vectors of dynamic objects visible to a vehicle. Similar to prior work [55], which uses an affinity matrix to associate objects across frames, ARC uses an affinity-based approach. It computes the affinity score between occupied cells in consecutive frames. We define the affinity score as

$$T(C_{t-1}, C_t) = w_1 \cdot L_2(C_{t-1}, C_t)^{-1} + w_2 \cdot \left|\rho_{c_{t-1}} - \rho_{c_t}\right|.$$

Here, C_{t-1} and C_t are the centroids of occupied cells in two consecutive frames. The term $L_2(C_{t-1}, C_t)^{-1}$ measures the inverse Euclidean distance between the centroids. The term $\left|\rho_{c_{t-1}} - \rho_{c_t}\right|$ measures the absolute difference in point density. The weights w_1 and w_2 control the contribution of each term. For each cell in the current frame, ARC finds the cell in the previous frame with the highest affinity score. This matches cells across frames. By tracking occupied cells and their point densities over consecutive frames, we build an affinity matrix. We then use this matrix to compute motion vectors for all occupied cells.

At this stage, every vehicle has two types of cells: a) object cells (cells with dynamic objects) and b) blind spots. With each object cell, ARC associates a motion vector and point density (computed



(a) Producer and its voxel map (b) Consumer and its voxel map (c) Consumer fused results

Figure 10: Voxel maps and grid cells for the producer (a), the consumer (b), and the fused object points at the consumer (c).

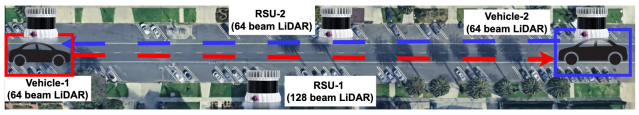


Figure 11: Real-world data collection setup with two vehicles and roadside units (RSUs): one RSU is equipped with a 128-beam LiDAR, while the other RSU and both vehicles use 64-beam LiDAR sensors.

in §3.1). ARC embeds this information into the voxel map (Fig. 10). The voxel map is built on top of the shared 3D grid. Instead of storing the 3D location of every cell, the voxel map stores the cell index in the grid. ARC builds two lists of indices (i.e., one for blind spot cells and one for object cells). The list for object cells also stores the motion vector and point density (Fig. 10a and Fig. 10b). Every vehicle broadcasts its voxel map to others at the intersection.

Vehicles receive voxel maps from others. For each blind spot cell, the vehicle checks which producers have that cell as an object cell with the desired point density. For example, in Fig. 10b, the vehicle has blind spot cells 6,7. It finds a producer vehicle (Fig. 10a) for which these cells are object cells. The consumer vehicle requests these points, the producer extracts them, and shares them. Finally, the consumer applies motion compensation and merges the received points with its own object cells (Fig. 10c).

4 Evaluation

4.1 Methodology

Implementation. We implemented ARC in C++. To build the 3D map, we used Fast-LIO2 [49]. Vehicles localize in this map using Normal Distribution Transform (NDT [6]). We used the Point Cloud Library (PCL) [38] for point cloud manipulation operations. PCL supports CUDA acceleration. ARC is 3,500 lines of C++ code. We evaluated ARC on a laptop with a 13th-generation Core i9 CPU and an NVIDIA GeForce RTX 4070.

Real-world Traces. We built a testbed to collect real-world traces using four LiDARs, two mounted on roadside units and two on moving vehicles, as shown in Fig. 11. There was other vehicular and pedestrian traffic present as we collected the data. One of the roadside units was equipped with a 128-beam LiDAR, whereas the other roadside unit and the vehicles were equipped with 64-beam

Figure 12: Bird's-eye view of a CARLA intersection with 40 vehicles, all participating in cooperative perception, each equipped with a 32-beam LiDAR (modeled after the Velodyne VLP-32C).

LiDARs. Although we collected a large corpus of data, we manually aligned 400 point clouds for accuracy evaluations.

Synthetic Traces. We used CARLA [16], an industry-standard photorealistic simulator, to generate synthetic LiDAR traces. CARLA allows flexible simulation of diverse traffic conditions and sensor configurations. Moreover, CARLA provides ground-truth point cloud alignments, enabling a comprehensive evaluation of accuracy.

Evaluation Metrics. In our evaluations, we measure the latency, accuracy, and bandwidth consumption of ARC per frame. Latency is the time from when a vehicle captures a LiDAR point cloud to when it fuses it with points received from others. This consists of compute latency and network latency. We measure only the compute latency, which is the time required to run ARC's algorithms. Network latency is the time it takes for data to travel from one vehicle to another. Network latency is more difficult to evaluate due to the scale of our experiments (*i.e.,* 40+ vehicles). Instead, we evaluate bandwidth consumption. This captures the amount of data vehicles generate to share over the network for cooperative perception.

For accuracy, we evaluate the alignment accuracy. We compute the relative translational error (RTE) and relative rotational error

Approach	RTE (cm)			RRE (°)		
	Mean	*p95*	*p99*	*Mean*	*p95*	*p99*
AutoCast	13.1	27.4	38.14	0.2	0.38	0.47
ARC	2.0	4.87	6.4	0.1	0.14	0.16

Table 1: Comparison of alignment accuracy (RTE and RRE) between ARC and AutoCast on CARLA simulation traces, reported as mean and at the 95th (*p95*) and 99th (*p99*) percentiles.

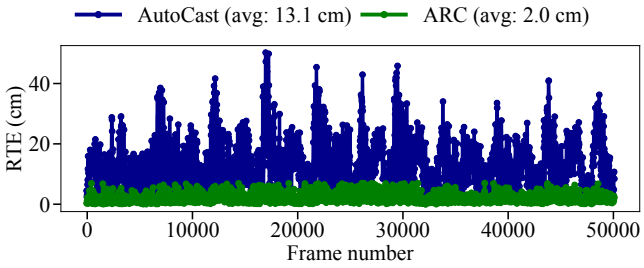


Figure 13: Comparison of ARC and AutoCast RTE on CARLA traces, showing an order-of-magnitude RTE reduction with ARC.

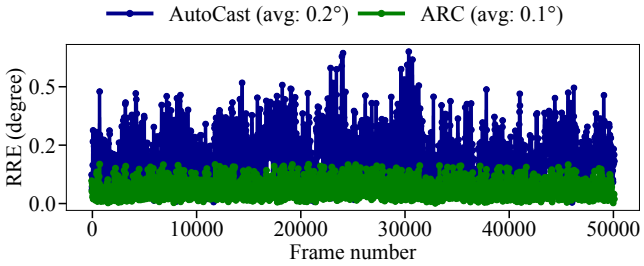


Figure 14: Comparison of ARC and AutoCast RRE on CARLA traces, showing that ARC reduces RRE compared to AutoCast.

(RRE) [18]. Both are root mean square errors between the estimated and ground-truth transformations, with lower values indicating higher accuracy. RTE is $||t - t_g||2$, where t is the translation vector estimated by ARC and tg is the ground truth. RRE is $\Sigma_{i=1}^{3}|angle(i)|$, where the angles are extracted from F $(R_g^{-1} \cdot R)$. Here, R_g and R denote the ground-truth and estimated rotation matrices, respectively. The function $F(.)$ converts a matrix into three Euler angles.

Baselines. Two recent vehicle-to-vehicle cooperative perception approaches are AutoCast [34] and RAO [55]. AutoCast uses a 3D map for alignment, whereas RAO relies on GPS/IMU. GPS/IMU-based alignment is known to introduce higher errors [19]. Therefore, we compare ARC with AutoCast as the representative baseline.

4.2 End-to-end Experiments: Accuracy

In this section, we evaluate ARC's accuracy in aligning point clouds.

CARLA Traces. To evaluate accuracy, we simulated vehicular traffic at an intersection in CARLA with 40 vehicles (Fig. 12). All vehicles participate in cooperative perception. Each vehicle is equipped with a 32-beam LiDAR (modeling the Velodyne VLP-32C [24]). We also mounted a roadside unit with the same LiDAR. From this experiment, we recorded 6,000 point clouds (150 per vehicle). We then aligned all vehicles with each other, resulting in 50,000 unique

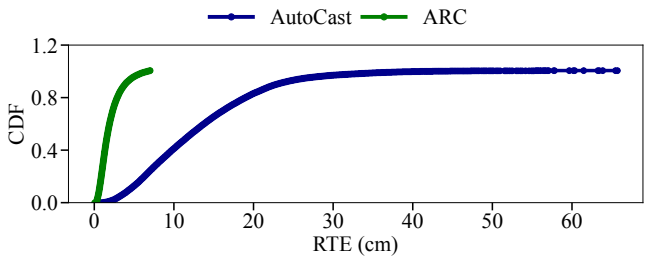


Figure 15: RTE CDFs for ARC and AutoCast on the CARLA traces.

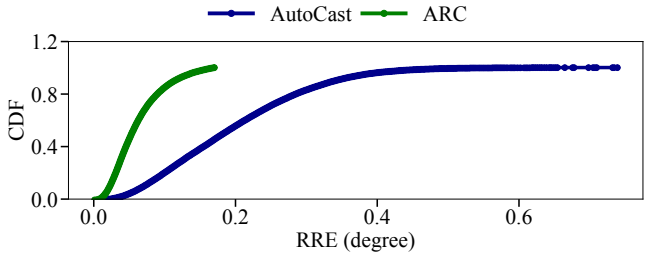


Figure 16: RRE CDFs for ARC and AutoCast on the CARLA traces.

alignments. We used ARC and AutoCast [34] to estimate the transformation matrices for all pairs. Recall that AutoCast [34] uses a 3D map to align vehicles. *On average, ARC improves accuracy (Fig. 13 and Fig. 14) by an order of magnitude compared to AutoCast.*

The average RTE for ARC is 2 cm, whereas it is 13 cm for AutoCast. The average RRE for ARC is 0.1°, compared to 0.2° for AutoCast. In the CDF plots (Fig. 15 and Fig. 16), ARC achieves significantly tighter alignment error distributions for both RTE and RRE, with the entire error range bounded within 7 cm and 0.15°, compared to 67 cm and 0.8° for AutoCast. As reported in Tbl. 1, ARC consistently achieves substantially lower tail errors compared to AutoCast. Specifically, the 95th and 99th percentile RTE values are reduced from 27 cm and 38 cm to 5 cm and 6.5 cm, while the 95th and 99th percentile RRE values improve from 0.4° and 0.5° to 0.14° and 0.16°, respectively. The improvement in alignment accuracy is due to ARC's use of an anchor point cloud for alignment, which does not accumulate noise over time. In contrast, the 3D map used by AutoCast can accumulate mapping errors due to sensor noise, inaccurate localization, and the presence of dynamic objects. These, in turn, affect the accuracy of the estimated transformation matrices.

Real-world Traces. We also evaluated ARC's alignment accuracy on real-world LiDAR traces from our testbed, which consists of two vehicles and two stationary roadside units. To generate ground truth, we manually aligned point clouds (with a large overlap) using CloudCompare [1]. With an average RTE and RRE of 6.4 cm and 0.34°, ARC demonstrates up to a 2× improvement in alignment accuracy compared to AutoCast (Fig. 17 and Fig. 18). ARC achieves significantly lower tail errors compared to AutoCast. The 95th and 99th percentile RTE values decrease from 34 cm and 48 cm to 10.7 cm and 11 cm, while the 95th and 99th percentile RRE values improve from 1.22° and 1.58° to 0.53° and 0.55° (Tbl. 2), respectively. These numbers are slightly higher than those from the synthetic traces, reflecting the increased complexity of real-world environments.

Approach	RTE (cm)			RRE (°)		
	Mean	*p95*	*p99*	*Mean*	*p95*	*p99*
AutoCast	13.30	34.3	48.0	0.55	1.22	1.58
ARC	6.40	10.7	11.0	0.34	0.53	0.55

Table 2: Accuracy of ARC and AutoCast on the real-world traces.

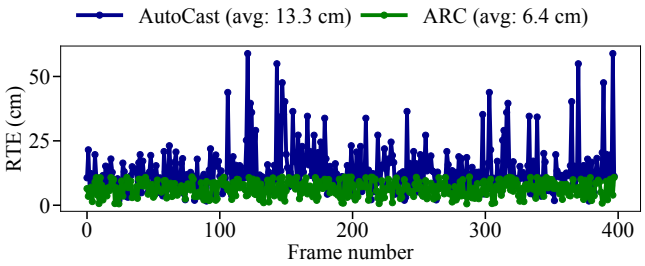


Figure 17: Comparison of ARC and AutoCast RTE on real-world traces, showing an order-of-magnitude RTE reduction with ARC.

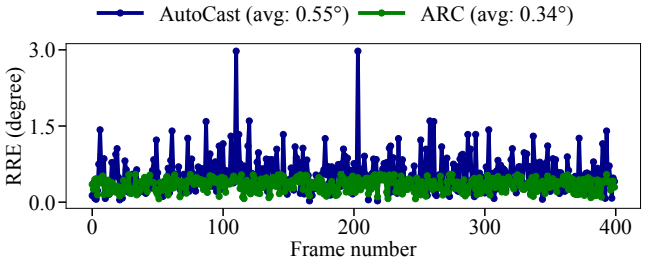


Figure 18: Comparison of ARC and AutoCast RRE on real-world traces, showing that ARC reduces RRE compared to AutoCast.

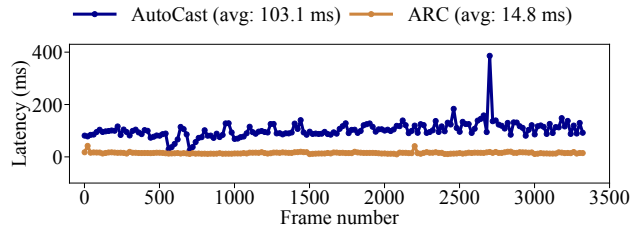


Figure 19: Comparison of ARC and AutoCast latency on CARLA traces, showing an order-of-magnitude reduction with ARC.

4.3 End-to-end Experiments: Latency

Next, we measure ARC's end-to-end compute latency. It is the time a vehicle takes to prepare its own data for sending, plus the time needed to process received data and fuse it with its own point cloud.

We measure the compute latency of AutoCast and ARC for each vehicle on the 40-vehicle CARLA dataset. We evaluate both approaches on the same compute platform as described in §4.1. For a fair comparison, we exclude the latency for path planning and collision avoidance in AutoCast. *Even so, ARC is an order of magnitude faster than AutoCast.* The average latency for ARC (Fig. 19, brown) is 14.8 ms, whereas for AutoCast (Fig. 19, blue), this is 103 ms. With an end-to-end compute latency of approximately 15 ms, ARC provides network and downstream modules (perception, planning, and

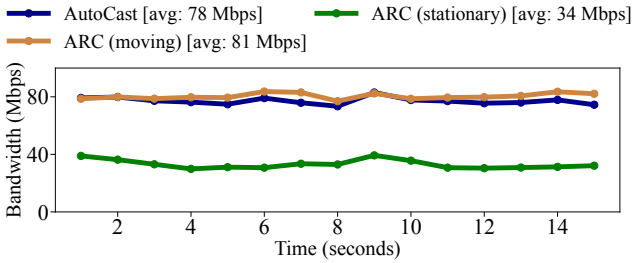


Figure 20: Comparison of ARC and AutoCast: cumulative bandwidth consumption for 40 vehicles on CARLA traces.

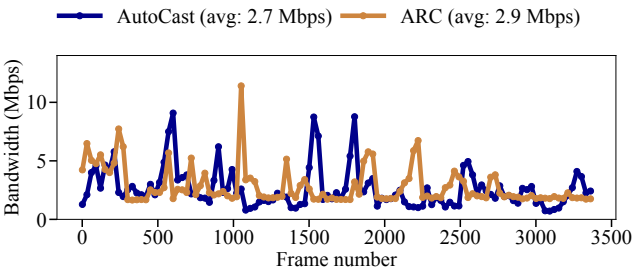


Figure 21: Comparison of ARC and AutoCast: per-vehicle bandwidth consumption on CARLA traces.

control [32]) roughly 85 ms to process the fused point cloud. In contrast, AutoCast generates the fused point cloud in 103 ms, so by the time downstream modules consume it, the data is already stale.

ARC ensures low latency even with additional alignment operations for accuracy for several reasons. Alignment is efficient because it aligns only overlapping regions. Furthermore, ARC reduces alignment overhead by reusing computations from overlap extraction to calculate vehicle blind spots. In contrast, AutoCast relies on CPU-intensive computations, whereas ARC leverages parallelized operations to fully utilize the GPU.

We provide the latency breakdown for all components of ARC in Tbl. 3. Localization latency refers to the time a vehicle takes to use NDT to align with the 3D map. Overlap estimation latency is the time required for a vehicle to estimate the overlap between its point cloud and the anchor point cloud. Anchor alignment latency is the time needed for a vehicle to align its overlapping region with the anchor. Producer processing latency involves identifying the objects requested by the consumer and sharing them. Consumer processing is the most computationally expensive module. Its latency includes the time needed for object extraction, blind spot detection, object cell estimation, motion vector estimation, and producer assignment.

We also evaluated the compute latency of ARC on real-world traces. The average latency per frame is 20 ms, slightly higher than the 15 ms measured using synthetic traces. For synthetic traces, we used a sparser LiDAR setup (32 beams instead of 64 or 128). We analyze the effect of LiDAR channel count on compute latency in §4.6. In our real-world deployment, both LiDARs are 64- or 128-beam, which explains the increased latency. On average, ARC takes 20 ms per frame across all four agents.

We could not measure AutoCast's compute latency on the real-world dataset because it requires access to the vehicle's path-planning

Process (on GPU)	Latency (ms)
Localization	2.1
Overlap estimation	0.3
Anchor alignment	2.0
Producer processing	2.0
Consumer processing	8.4
Total	14.8

Table 3: ARC's compute latency breakdown for the CARLA traces.

Approach	3D Points (MB)	Control (MB)	Total (MB)
AutoCast [34]	0.87	*0.1*	*0.97*
ARC	*0.44*	0.58	1.02

Table 4: Comparison of per-frame control and 3D (LiDAR points) data transmitted by ARC and AutoCast using CARLA traces.

Approach	Error		Lat. (ms)	BW (Mbps)
	RTE (cm)	*RRE (°)*		
Indirect	13.1	0.20	*2.10*	78
Direct	*1.5*	*0.05*	∞	∞
ARC (moving)	2.0	0.10	14.8	81
ARC (stationary)	2.5	0.10	15.5	*34*

Table 5: Performance of various alignment strategies of ARC.

module, which raises safety concerns. Furthermore, the open-source implementation of AutoCast supports only CARLA simulations and does not support real-world deployment.

4.4 End-to-end Experiments: Network

Next, we measure the networking requirements of ARC by quantifying the total bandwidth (Mbps) consumed collectively by all vehicles. We evaluate this metric on both synthetic and real-world traces. On average, ARC requires 81 Mbps (Fig. 20, brown) for 40 vehicles. In contrast, AutoCast requires 78 Mbps (Fig. 20, blue). The bandwidth requirement of ARC is higher than that of AutoCast since the anchor broadcasts its point cloud in every frame. When the anchor is a stationary roadside sensor, this requirement drops to 34 Mbps (Fig. 20, green). This is because the anchor shares its reference point cloud with vehicles only once rather than periodically. AutoCast's higher bandwidth is due to redundant data sharing, which becomes more severe as the number of vehicles increases.

We provide a communication cost breakdown for both AutoCast and ARC (Tbl. 4). ARC reduces per-frame 3D point data sharing from 0.87 MB to 0.44 MB by eliminating redundant transmissions. It incurs higher control overhead (0.58 MB vs. 0.1 MB) due to per-frame anchor point cloud broadcasts. Nonetheless, the total data produced remains comparable (1.02 MB vs. 0.97 MB).

Additionally, we calculate per-vehicle bandwidth requirement. For each vehicle, the required bandwidth is 2.9 Mbps for ARC (Fig. 21, brown) and 2.7 Mbps for AutoCast (Fig. 21, blue) on synthetic traces. We exclude frames in which no data is shared among vehicles, as they do not contribute to cooperative perception. For real-world traces, each vehicle uses roughly 0.5 Mbps due to fewer vehicles sharing data compared to the synthetic traces. We were unable to evaluate bandwidth for AutoCast in this case, as it required access to the vehicle's motion planning module.

Alignment Technique	Error		Lat. (ms)
	RTE (cm)	*RRE (°)*	
FGR	448	4.0	230
SAC-IA	900	30	280
R-PointHop	733	45	600
Go-ICP	327	10	730
ICP (GPS)	536	16	185
ICP (3D map)	2.3	0.1	25
ARC	*2.0*	*0.1*	*2.7*

Table 6: Comparison of alignment accuracy and latency between ARC and scan-matching, feature-based, and learning-based techniques, showing an order-of-magnitude reduction in RTE and compute latency with ARC.

4.5 Ablation Studies

In this section, we perform ablation studies to quantify ARC's design decisions for point cloud alignment and data sharing.

Alignment Strategies. In Tbl. 5, we evaluate the alignment accuracy, compute latency, and bandwidth requirements of different 3D point cloud alignment strategies for cooperative perception. These include indirect alignment of point clouds using a 3D map (Indirect), direct alignment of point clouds (Direct), and the use of ARC to align point clouds with moving and stationary anchors. Ideally, to achieve the highest alignment accuracy, two point clouds should be directly aligned. However, this approach is not practical due to computational and network constraints. If a vehicle needs to fuse its point cloud with those from multiple vehicles, it must perform several alignment operations per frame, which can quickly exhaust compute resources. Network bandwidth also becomes a bottleneck, as each vehicle would need to broadcast its entire point cloud.

The lowest latency is achieved with indirect alignment because once both vehicles are aligned to the 3D map, they are also aligned with one another. However, this approach can result in high alignment errors. ARC navigates this tradeoff to simultaneously achieve low alignment error, low compute latency, and low bandwidth requirements. ARC's point cloud alignment is fast because it aligns only overlapping regions. In addition, bandwidth requirements are lower because it shares only relevant information with other vehicles rather than the entire point cloud. The bandwidth requirement for ARC with a stationary anchor is the lowest, as it does not need to broadcast the anchor point cloud every frame. Finally, alignment error is lower than that of indirect alignment and comparable to direct alignment because ARC aligns against an anchor point cloud, rather than a 3D map, which accumulates errors over time.

Overlap-aware Alignment. In this section, we evaluate ARC's ability to accurately and efficiently align vehicle point clouds to the anchor using overlap-aware alignment. For this, we use a CARLA dataset consisting of 150 vehicle-anchor point cloud pairs. We align each vehicle point cloud to the anchor. In Tbl. 6, we compare ARC's alignment accuracy and latency against scan matching, feature matching, and learning-based techniques. Feature-based techniques (FGR [58] and SAC-IA [37]) exhibit high alignment errors because they cannot reliably match point cloud features captured from diverse viewpoints. R-PointHop [20], a recent learning-based technique using hierarchical features for correspondences, is also unable to accurately align sparser vehicle point clouds. The alignment error for R-PointHop is on the order of 700 cm.

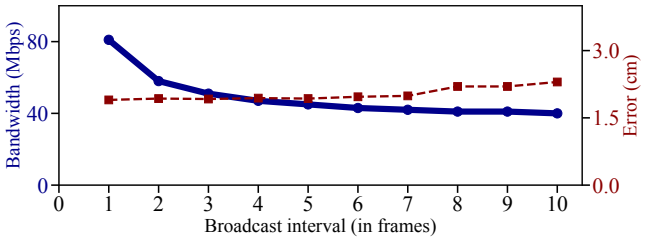


Figure 22: Impact of anchor broadcast frequency on ARC's alignment error and bandwidth requirements.

Scan-matching techniques such as ICP [36] are sensitive to initialization. Poor initial guesses (such as those provided by GPS) result in inaccurate alignments and high latency. Variations of ICP that are less sensitive to initial guesses (e.g., GO-ICP [50]) can be compute-intensive (730 ms latency). Although 3D maps provide more precise initial guesses, aligning entire point clouds remains compute-intensive (latency on the order of 25 ms).

In contrast, ARC achieves fast alignment with a latency of only 3 ms and high accuracy, with an RTE of just 2 cm. Its speed and accuracy result from aligning only overlapping regions and leveraging a relatively accurate initial guess.

Anchor Broadcast Frequency. We also evaluate how reducing the anchor's broadcast frequency impacts alignment accuracy and bandwidth. Using the CARLA dataset (§4.2), we vary the broadcast interval from every frame to once every 10 frames (Fig. 22). Alignment error increases only slightly, from 1.9 cm to 2.3 cm (Fig. 22, brown), while bandwidth usage drops significantly, from 81 Mbps to 40 Mbps (Fig. 22, blue). These results show that our approach can adapt to lower network bandwidth by reducing broadcast frequency, achieving up to 50% savings with minimal loss in alignment accuracy. The tradeoff is that if a vehicle misses a broadcast, it must wait several frames before receiving the next anchor point cloud, unlike the case with per-frame broadcasting.

4.6 Sensitivity Analysis

We perform a sensitivity analysis of anchor point cloud occlusion, traffic density, vehicle speed, and sensor configuration on ARC.

Anchor Point Cloud Occlusion. ARC's alignment accuracy degrades as occlusion of the anchor's point cloud increases. Using our CARLA dataset (§4.2), we simulate occlusion levels from 10% to 90% by obscuring portions of the anchor's point cloud. As shown in Fig. 23, alignment error exceeds 10 cm when occlusion is above 50% and grows to meters beyond 80%. In our dataset, with 40 vehicles at the intersection, the average occlusion is 17% (corresponding to a 2 cm error), with 95th and 99th percentiles at 24% (2.3 cm) and 27% (2.7 cm). This demonstrates that even at higher occlusion levels, up to 27%, ARC maintains an alignment error below 3 cm.

Traffic Density. To evaluate the effect of traffic density, we simulated three traffic conditions in CARLA. For low traffic, 10–15 vehicles; for medium, 15–30; and for high traffic, 30–40 vehicles at the intersection. Results are summarized in Tbl. 7. As traffic density increases, the required bandwidth for ARC rises from 56 Mbps to 81 Mbps, because more vehicles create blind spots and exchange more data. Compute latency remains relatively constant, as point cloud alignment and the decision of what data to share are largely independent of the number of surrounding vehicles. The alignment error increases only marginally with more vehicles.

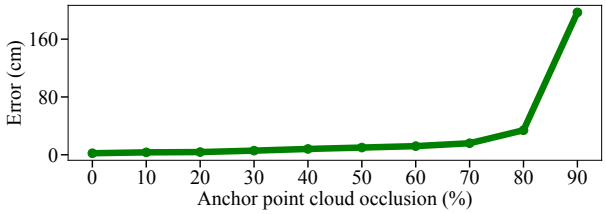


Figure 23: Impact of anchor occlusion on ARC's accuracy.

Traffic Density	Error		Lat. (ms)	BW (Mbps)
	RTE (cm)	*RRE (°)*		
Low (10 - 15)	1.40	0.05	14.5	56
Medium (15 - 30)	1.75	0.06	14.2	67
High (30 - 40)	2.00	0.10	14.8	81

Table 7: Effect of traffic density on ARC's performance.

Vehicle Speed	Error		Lat. (ms)	BW (Mbps)
	RTE (cm)	*RRE (°)*		
Low (25 km/hr)	1.26	0.03	13.4	56
Medium (50 km/hr)	1.30	0.03	13.6	59
High (75 km/hr)	1.40	0.05	14.5	59

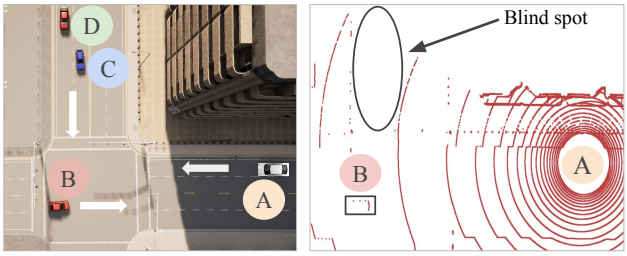
Table 8: Effect of varying vehicle speed on ARC's performance.

LiDAR Channels	Error		Lat. (ms)	BW (Mbps)
	RTE (cm)	*RRE (°)*		
32	1.4	0.05	14.5	56
64	1.3	0.03	17.6	71
128	1.1	0.02	22.5	86

Table 9: Effect of LiDAR channels on ARC's performance.

Vehicle Speed. To quantify the impact of vehicle speed, we collected multiple traces in CARLA with vehicles driving at different speeds while keeping traffic density constant (low). Results are summarized in Tbl. 8. Low corresponds to an average speed of 25 km/hr, medium to 50 km/hr, and high to 75 km/hr. The results indicate that ARC is agnostic to vehicle speed, with minimal impact on alignment accuracy, latency, or bandwidth consumption.

LiDAR Channels. In the final set of sensitivity analysis experiments, we evaluated ARC's performance with different LiDAR configurations. We deployed 32-beam, 64-beam, and 128-beam LiDARs at the traffic intersection in CARLA, keeping traffic density constant (low). Results are summarized in Tbl. 9. Increasing the number of channels raises both compute latency and required network bandwidth. This is because ARC's anchor alignment, overlap estimation, and object extraction are sensitive to the number of points in the point cloud. Higher-resolution LiDARs produce denser point clouds, increasing computational cost. Dense point clouds mean denser regions to share to compensate for blind spots, leading to higher network bandwidth requirements. Finally, denser point clouds lead to greater overlap, which improves alignment accuracy.



(a) A CARLA intersection (b) Vehicle A's point cloud

Figure 24: A CARLA scenario where a building on the right side of vehicle A causes occlusion. Vehicle C and vehicle D are not visible in vehicle A's point cloud.

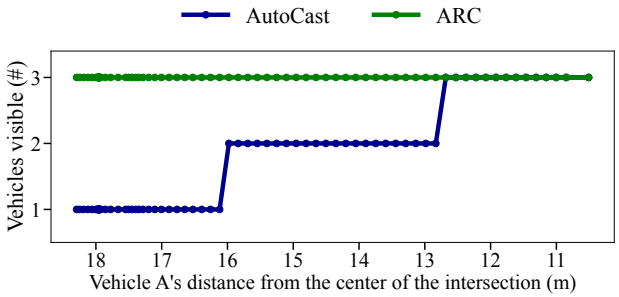


Figure 25: Comparison of AutoCast and ARC in the scenario shown in Fig. 24. It shows the number of vehicles visible to vehicle A in each frame as a function of its distance from the intersection center.

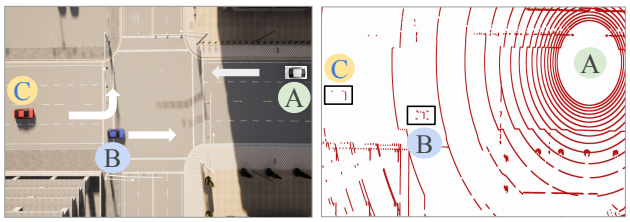
4.7 Application-Level Benefits

Improved Visibility. We demonstrate ARC's advantage over AutoCast in detecting vehicles in blind spots. Here, visibility coverage is defined as the number of vehicles detected within a vehicle's perception range, regardless of line-of-sight limitations.

In Fig. 24a, we show a CARLA scenario with four vehicles (A, B, C, and D) approaching an intersection. Each vehicle is equipped with a 32-beam LiDAR (120 m range). Fig. 24b shows vehicle A's LiDAR point cloud, where vehicles C and D are missing due to a blind spot created by the building on the right side of vehicle A.

Fig. 25 shows the number of vehicles detected by vehicle A as a function of its distance from the center of the intersection. Using AutoCast, vehicle A detects C only at 16 m and D at 12 m (Fig. 25, blue). In contrast, with ARC, vehicle A maintains visibility of all three vehicles throughout its trajectory (Fig. 25, green). This is because AutoCast uses ray-casting to find blind spots and shares objects hidden by other vehicles. However, it ignores occlusions caused by static objects such as buildings. ARC, on the other hand, identifies blind spots as low-density regions in the point cloud and enables vehicles to share data to fill these areas. This allows vehicles to *see* objects occluded by either dynamic or static objects.

Improved Point Density. In this section, we demonstrate that ARC improves point density (number of points per unit volume) compared to AutoCast. To evaluate this, we simulate a CARLA scenario (Fig. 26a) in which vehicles A, B, and C approach an intersection. Each vehicle is equipped with a 32-beam LiDAR (120 m range). We compare the mean point density of vehicles visible to A



(a) A CARLA intersection (b) Vehicle A's point cloud

Figure 26: A CARLA scenario where vehicles A, B, and C are at an intersection. In vehicle A's point cloud, vehicles B and C are visible.

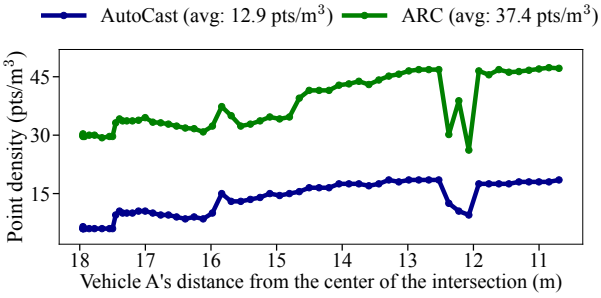


Figure 27: Comparison of AutoCast and ARC in the scenario shown in Fig. 26. It shows the mean point density of objects visible to vehicle A as a function of its distance from the intersection.

(Fig. 26b) for both AutoCast and ARC. Fig. 27 presents this comparison, showing the mean point density of vehicles visible to A as a function of A's distance from the center of the intersection.

AutoCast activates perception data exchange only in the presence of occlusions. In contrast, ARC targets regions with low point density, including blind spots and other sparse areas. This leads to more effective data sharing and, consequently, increases the point density of the resulting point clouds. As shown in Fig. 27, ARC improves the mean point density by a factor of 3 compared to AutoCast. As a result, downstream perception modules, such as object detection [52] and semantic segmentation [29], receive point clouds with higher point density, which improves their accuracy [14].

5 Related Work

Cooperative Perception. Cooperative perception enables vehicles to extend their sensing range beyond what their own sensors can capture. It mainly follows two approaches: vehicle-to-vehicle (V2V) [27, 33, 34, 55] and vehicle-to-infrastructure (V2I) [12, 18, 39, 51, 57]. V2V is appealing because it does not rely on roadside infrastructure, but poses significant scalability challenges. Some V2V approaches, such as AutoCast [34] and RAO [55], exchange raw point clouds, while others, including OPV2V [48], F-Cooper [10], Core [45], V2V-LLM [13], and InterCoop [47], share processed data. These approaches aim to balance latency, accuracy, and scalability, but often do so at the expense of one another. In contrast, ARC is a V2V cooperative perception system that achieves high accuracy and scalability without compromising latency.

Sensor Data Alignment. To fuse external sensor data, a vehicle must align it to its own coordinate system: either directly using

algorithms like ICP [36] or indirectly through a shared 3D map or GPS. Direct alignment techniques are accurate [21] but not scalable. Indirect methods scale better but suffer from lower accuracy. Most existing approaches rely on 3D maps [33, 34] or GPS [10, 48, 55] for indirect alignment, both of which provide limited alignment accuracy. In contrast, ARC uses a single point cloud for indirect alignment to achieve scalability and accuracy simultaneously.

Sharing Relevant Information. Cellular Vehicle-to-Everything (C-V2X) [53] allows vehicles to communicate with each other and nearby infrastructure. It supports data rates up to 100 Mbps [54], but sharing raw sensor data can exhaust the network. Methods like AVR [33] exchange raw data but do not scale. EMP [56] reduces data volume by performing spatial reasoning at the edge, yet its high latency limits scalability. More recent methods [34, 55] share only relevant information, such as occluded objects. AutoCast [34] reduces bandwidth usage but can produce inaccurate and redundant results. RAO [55] improves accuracy and avoids redundancy, but introduces extra computation. In contrast, ARC reuses computations across modules to perform fast and accurate spatial reasoning. This allows scalable information sharing without adding latency.

6 Discussion and Future Work

Impact of Network Latency. In practice, ARC's end-to-end latency consists of compute latency (time to run ARC's algorithms) and network latency (time to transmit data over the wireless network). Directly measuring network latency across tens of communicating vehicles was infeasible due to safety and scalability concerns, so our evaluation focuses on compute latency. On average, ARC's compute latency is 20 ms per point cloud, leaving 80 ms within the 100 ms reaction window for safe autonomous operation [26].

For network evaluations, we quantified the wireless bandwidth required to run ARC. On average, ARC needs 40 Mbps of sustained network bandwidth. Communication technologies such as 5G Vehicle-to-Everything (5G-V2X [4]), which support both vehicle-to-vehicle and vehicle-to-infrastructure communication, can theoretically provide up to 100 Mbps throughput with sub-10 ms one-way latency [41]. Field deployments typically demonstrate 30–40 Mbps sustained throughput [8]. These figures suggest that ARC's bandwidth and latency demands are within the operational range of 5G-V2X systems. However, we note that in practice, network performance depends on several factors, including the wireless environment, technology stack, and network density. At higher traffic densities, contention and scheduling overheads can increase latency. Though we leave a more thorough evaluation to future work, below, we qualitatively describe the effects of network latency on ARC.

Anchor selection exchanges small control messages; hence, network impact is minimal. In contrast, anchor broadcasts involve large point clouds that consume significant bandwidth and can delay the start of alignment if the network is congested. Reducing the frequency of broadcasts (§4.5) helps mitigate this effect but does not eliminate it, especially as fleet size grows. Future network technologies, such as mmWave [43], which offer higher data rates and improved scheduling efficiency, are expected to further reduce latency, enabling more timely and scalable operation of ARC.

Adding Multiple Anchors. In §4.2, we show that ARC achieves 2 cm accuracy with a single anchor. Deploying multiple anchors could translate to a larger overlap between anchors and other vehicles. Moreover, it adds an additional layer of robustness to ARC. However, this comes at the cost of increased network load and complexity, as all anchors must broadcast their point clouds and perform mutual alignment. We leave a thorough evaluation to future work.

7 Conclusions

In this paper, we presented ARC, a multi-vehicle cooperative perception framework that achieves high accuracy and low latency while efficiently scaling to many vehicles. We evaluated it against state-of-the-art on two datasets: one collected in the real world using 64- and 128-beam LiDARs, and another logged from CARLA. The results show an order-of-magnitude improvement in latency and accuracy. Specifically, ARC provides alignment accuracy below 7 cm with an end-to-end compute latency of 20 ms. Moreover, it achieves higher point density and improved visibility coverage compared to prior work. Finally, sensitivity analysis confirms its robustness across traffic density, vehicle speed, and LiDAR configuration.

Acknowledgments

We thank the anonymous reviewers and shepherd for their insightful comments. This material is based upon work supported by the U.S. National Science Foundation (award number: CNS-2348461).

References

[1] 2023. *CloudCompare.* https://www.danielgm.net/cc/
[2] 2025. Autoware: Open-source software for self-driving vehicles. https://github.com/autowarefoundation/autoware_ai.
[3] 2025. Baidu Apollo. https://www.apollo.auto/apollo-self-driving.
[4] 5G Automotive Association. 2024. *5G-V2X Direct Communication Evaluation Approach: An Automotive Analysis.* White Paper. 5G Automotive Association (5GAA). https://5gaa.org/content/uploads/2024/07/5gaa-wi-nr-v2x-eval.pdf
[5] 5G-PPP. 2015. 5G Automotive Vision. https://5g-ppp.eu/wp-content/uploads/2014/02/5G-PPP-White-Paper-on-Automotive-Vertical-Sectors.pdf. Accessed: Oct. 23, 2025.
[6] Naoki Akai, Luis Yoichi Morales, Eijiro Takeuchi, Yuki Yoshihara, and Yoshiki Ninomiya. 2017. Robust localization using 3D NDT scan matching with experimentally determined uncertainty and road marker matching. In *2017 IEEE Intelligent Vehicles Symposium (IV).* IEEE, 1356–1363.
[7] An Open Source Self-Driving Car. 2023. An Open Source Self-Driving Car. www.udacity.com/self-driving-car.
[8] Waqar Anwar, Andreas Traßl, Norman Franchi, and Gerhard Fettweis. 2019. On the Reliability of NR-V2X and IEEE 802.11 bd. In *2019 IEEE 30th annual international symposium on personal, indoor and mobile radio communications (PIMRC).* IEEE, 1–7.
[9] Automotive Research News. 2025. How Cameras are Paving the Way for Safer Autonomous Driving. https://automotiveresearchnews.com/autonomous-vehicles-av/how-cameras-are-paving-the-way-for-safer-autonomous-driving/ Accessed: 2025-07-02.
[10] Qi Chen, Xu Ma, Sihai Tang, Jingda Guo, Qing Yang, and Song Fu. 2019. F-cooper: Feature based cooperative perception for autonomous vehicle edge computing system using 3D point clouds. In *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing.* 88–100.
[11] Xieyuanli Chen, Andres Milioto, Emanuele Palazzolo, Philippe Giguere, Jens Behley, and Cyrill Stachniss. 2019. Suma++: Efficient lidar-based semantic slam. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS).* IEEE, 4530–4537.
[12] Ziming Chen, Yifeng Shi, and Jinrang Jia. 2023. TransIFF: An Instance-Level Feature Fusion Framework for Vehicle-Infrastructure Cooperative 3D Detection with Transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision.* 18205–18214.
[13] Hsu-kuang Chiu, Ryo Hachiuma, Chien-Yi Wang, Stephen F Smith, Yu-Chiang Frank Wang, and Min-Hung Chen. 2025. V2v-llm: Vehicle-to-vehicle cooperative autonomous driving with multi-modal large language models. *arXiv preprint arXiv:2502.09980* (2025).
[14] Eduardo R Corral-Soto, Alaap Grandhi, Yannis Y He, Mrigank Rochan, and Bingbing Liu. 2023. Improving lidar 3d object detection via range-based point cloud density optimization. *arXiv preprint arXiv:2306.05663* (2023).

[15] Cruise. 2019. Cruise. https://medium.com/cruise/hd-maps-self-driving-cars-b6444720021c.

[16] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. 2017. CARLA: An open urban driving simulator. *arXiv preprint arXiv:1711.03938* (2017).

[17] Martin A Fischler and Robert C Bolles. 1981. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Commun. ACM* 24, 6 (1981), 381–395.

[18] Yuze He, Li Ma, Zhehao Jiang, Yi Tang, and Guoliang Xing. 2021. VI-Eye: Semantic-Based 3D Point Cloud Registration for Infrastructure-Assisted Autonomous Driving. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*. 573–586.

[19] Yurong Jiang, Hang Qiu, Matthew McCartney, Gaurav Sukhatme, Marco Gruteser, Fan Bai, Donald Grimm, and Ramesh Govindan. 2015. Carloc: Precise positioning of automobiles. In *Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems*. 253–265.

[20] Pranav Kadam, Min Zhang, Shan Liu, and C-C Jay Kuo. 2022. R-pointhop: A green, accurate, and unsupervised point cloud registration method. *IEEE Transactions on Image Processing* 31 (2022), 2710–2725.

[21] Kaleem Nawaz Khan, Ali Khalid, Yash Turkar, Karthik Dantu, and Fawad Ahmad. 2024. VRF: Vehicle Road-side Point Cloud Fusion. In *Proceedings of the 22nd Annual International Conference on Mobile Systems, Applications and Services*. 547–560.

[22] Alex H Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. 2019. Pointpillars: Fast encoders for object detection from point clouds. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 12697–12705.

[23] Zhichao Li, Feng Wang, and Naiyan Wang. 2021. Lidar r-cnn: An efficient and universal 3d object detector. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 7546–7555.

[24] Velodyne LiDAR. 2020. Velodyne LiDAR. https://velodynelidar.com/.

[25] Hsien-I Lin, Muhammad Ahsan Fatwaddin Shodiq, An-Kai Jeng, and Chun-Wei Chang. 2025. An Efficient Large-Scale 3D Map Stitching Algorithm Using Automatic Overlapping Area Identification. *IEEE Access* (2025).

[26] Shih-Chieh Lin, Yunqi Zhang, Chang-Hong Hsu, Matt Skach, Md E Haque, Lingjia Tang, and Jason Mars. 2018. The architectural implications of autonomous driving: Constraints and acceleration. In *Proceedings of the twenty-third international conference on architectural support for programming languages and operating systems*. 751–766.

[27] Yunsheng Ma, Juanwu Lu, Can Cui, Sicheng Zhao, Xu Cao, Wenqian Ye, and Ziran Wang. 2024. MACP: Efficient model adaptation for cooperative perception. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 3373–3382.

[28] Andres Milioto, Ignacio Vizzo, Jens Behley, and Cyrill Stachniss. 2019. Rangenet++: Fast and accurate lidar semantic segmentation. In *2019 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 4213–4220.

[29] Andres Milioto, Ignacio Vizzo, Jens Behley, and Cyrill Stachniss. 2019. Rangenet++: Fast and accurate lidar semantic segmentation. In *2019 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 4213–4220.

[30] NHTSA. 2023. NHTSA | National Highway Traffic and Safety Authority. https://www.nhtsa.gov/.

[31] Inc. Ouster. 2025. Digital Lidar Sensors for Automation, Drones & Robotics. https://ouster.com/ Accessed: 2025-07-02.

[32] Scott Drew Pendleton, Hans Andersen, Xinxin Du, Xiaotong Shen, Malika Meghjani, You Hong Eng, Daniela Rus, and Marcelo H Ang. 2017. Perception, planning, control, and coordination for autonomous vehicles. *Machines* 5, 1 (2017), 6.

[33] Hang Qiu, Fawad Ahmad, Fan Bai, Marco Gruteser, and Ramesh Govindan. 2018. Avr: Augmented vehicular reality. In *Proceedings of the 16th Annual International Conference on Mobile Systems, Applications, and Services*. 81–95.

[34] Hang Qiu, Pohan Huang, Namo Asavisanu, Xiaochen Liu, Konstantinos Psounis, and Ramesh Govindan. 2022. AutoCast: Scalable Infrastructure-less Cooperative Perception for Distributed Collaborative Driving. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '22)*.

[35] Maurice Quach, Jiahao Pang, Dong Tian, Giuseppe Valenzise, and Frédéric Dufaux. 2022. Survey on deep learning-based point cloud compression. *Frontiers in Signal Processing* 2 (2022), 846972.

[36] Szymon Rusinkiewicz and Marc Levoy. 2001. Efficient variants of the ICP algorithm. In *Proceedings third international conference on 3-D digital imaging and modeling*. IEEE, 145–152.

[37] Radu Bogdan Rusu, Nico Blodow, and Michael Beetz. 2009. Fast Point Feature Histograms (FPFH) for 3D registration. In *2009 IEEE International Conference on Robotics and Automation, ICRA 2009, Kobe, Japan, May 12-17, 2009*. 3212–3217. doi:10.1109/ROBOT.2009.5152473

[38] Radu Bogdan Rusu and Steve Cousins. 2011. 3D is here: Point Cloud Library (PCL). *IEEE International Conference on Robotics and Automation (ICRA)* (2011), 1–4. doi:10.1109/ICRA.2011.5980567

[39] Shuyao Shi, Jiahe Cui, Zhehao Jiang, Zhenyu Yan, Guoliang Xing, Jianwei Niu, and Zhenchao Ouyang. 2022. VIPS: Real-Time Perception Fusion for Infrastructure-Assisted Autonomous Driving. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*. 133–146.

[40] Christina Suyong Shin, Weiwu Pang, Chuan Li, Fan Bai, Fawad Ahmad, Jeongyeup Paek, and Ramesh Govindan. 2024. RECAP: 3D Traffic Reconstruction. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 1252–1267.

[41] Steve Taranovich. 2021. Taking a Deep Dive into New V2X Architectures. Mouser Electronics. https://www.mouser.ca/applications/new-v2x-architectures/ Online; accessed 2026-02-12.

[42] Hesai Technology. 2024. OT128 Automotive-Grade 360° Long-Range Lidar. https://www.hesaitech.com/product/ot128/. https://www.hesaitech.com/product/ot128/ Accessed: 2025-10-26.

[43] Vutha Va, Takayuki Shimizu, Gaurav Bansal, Robert W Heath Jr, et al. 2016. Millimeter wave vehicular communications: A survey. *Foundations and Trends® in Networking* 10, 1 (2016), 1–113.

[44] Vehicle Empire. 2025. Understanding the Role of Radar in Autonomous Vehicles. https://vehicleempire.com/radar-in-autonomous-vehicles/ Accessed: 2025-07-02.

[45] Binglu Wang, Lei Zhang, Zhaozhong Wang, Yongqiang Zhao, and Tianfei Zhou. 2023. Core: Cooperative reconstruction for multi-agent perception. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 8710–8720.

[46] Miaohui Wang, Runnan Huang, Wuyuan Xie, Zhan Ma, and Siwei Ma. 2025. Compression Approaches for LiDAR Point Clouds and Beyond: A Survey. *ACM Transactions on Multimedia Computing, Communications and Applications* (2025).

[47] Wentao Wang, Haoran Xu, and Guang Tan. 2024. InterCoop: Spatio-Temporal Interaction Aware Cooperative Perception for Networked Vehicles. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 14443–14449.

[48] Runsheng Xu, Hao Xiang, Xin Xia, Xu Han, Jinlong Li, and Jiaqi Ma. 2022. Opv2v: An open benchmark dataset and fusion pipeline for perception with vehicle-to-vehicle communication. In *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2583–2589.

[49] Wei Xu, Yixi Cai, Dongjiao He, Jiarong Lin, and Fu Zhang. 2022. Fast-lio2: Fast direct lidar-inertial odometry. *IEEE Transactions on Robotics* 38, 4 (2022), 2053–2073.

[50] Jiaolong Yang, Hongdong Li, Dylan Campbell, and Yunde Jia. 2015. Go-ICP: A globally optimal solution to 3D ICP point-set registration. *IEEE transactions on pattern analysis and machine intelligence* 38, 11 (2015), 2241–2254.

[51] Sheng Yi, Hao Zhang, Feiyu Jin, Yiyang Hu, Rongzhen Li, and Kai Liu. 2024. V2ICooper: Toward Vehicle-to-Infrastructure Cooperative Perception with Spatiotemporal Asynchronous Fusion. In *International Conference on Wireless Artificial Intelligent Computing Systems and Applications*. Springer, 52–64.

[52] Zhenxun Yuan, Xiao Song, Lei Bai, Zhe Wang, and Wanli Ouyang. 2021. Temporal-channel transformer for 3d lidar-based video object detection for autonomous driving. *IEEE Transactions on Circuits and Systems for Video Technology* 32, 4 (2021), 2068–2078.

[53] Syed Adnan Yusuf, Arshad Khan, and Riad Souissi. 2024. Vehicle-to-everything (V2X) in the autonomous vehicles domain–A technical review of communication, sensor, and AI technologies for road user safety. *Transportation Research Interdisciplinary Perspectives* 23 (2024), 100980.

[54] Eduard Zadobrischi and Ștefan Havriliuc. 2024. Enhancing scalability of C-V2X and DSRC vehicular communication protocols with lora 2.4 GHz in the scenario of urban traffic systems. *Electronics* 13, 14 (2024), 2845.

[55] Qingzhao Zhang, Xumiao Zhang, Ruiyang Zhu, Fan Bai, Mohammad Naserian, and Z Morley Mao. 2023. Robust Real-time Multi-vehicle Collaboration on Asynchronous Sensors. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*. 1–15.

[56] Xumiao Zhang, Anlan Zhang, Jiachen Sun, Xiao Zhu, Y Ethan Guo, Feng Qian, and Z Morley Mao. 2021. EMP: edge-assisted multi-vehicle perception. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*. 545–558.

[57] Jiaru Zhong, Haibao Yu, Tianyi Zhu, Jiahui Xu, Wenxian Yang, Zaiqing Nie, and Chao Sun. 2024. Leveraging temporal contexts to enhance vehicle-infrastructure cooperative perception. *arXiv preprint arXiv:2408.10531* (2024).

[58] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. 2016. Fast global registration. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part II 14*. Springer, 766–782.

[59] Zoox. 2023. Zoox. https://zoox.com/journal/putting-our-robots-on-the-map/.